

1. Objective

Take the Student REST API you built in Lab 3 (Express.js or Spring Boot - whichever you used as your **full implementation** stack) and complete the full stack around it: replace the in-memory storage with a persistent **MongoDB Atlas** database, then build client applications on top of the same API - a complete CRUD web client in **React**, and equivalent read/create screens in an **Android** app. This mirrors the "full in one stack, equivalent in the other" approach you already used for Express.js vs Spring Boot in Lab 3, now applied to web vs mobile clients.

Full-stack flow for this lab:

Database (MongoDB Atlas) ↔ REST API (Express.js / Spring Boot, same project as Lab 3) ↔ React Web Client (full CRUD) **and** Android Client (List + Add)

2. Multi-Client / SOA Architecture

Both client applications talk to the **same REST API** - neither one connects to MongoDB directly. This is the core Web Services / SOA idea this lab is built around: a single backend service exposed over REST, consumed independently by multiple, unrelated clients.

Layer	Talks To	Never Talks To
React Client	REST API (HTTP/JSON)	MongoDB Atlas directly
Android Client	REST API (HTTP/JSON)	MongoDB Atlas directly
REST API	MongoDB Atlas (driver)	-

Discussion question (answer briefly in your README): **Why should React and Android access the REST service rather than connect directly to MongoDB?** Think about security, validation, coupling, and what happens if the database technology changes later.

3. Part A - Database Integration (Backend)

Replace the in-memory array/list from Lab 3 with a real MongoDB Atlas database, keeping the same endpoints, request/response shape and status codes.

- Create a free MongoDB Atlas cluster (M0 tier); create a database user and allow network access for your IP (or 0.0.0.0/0 for lab purposes).
- Install a driver in your Lab 3 project: **Mongoose** for Express.js, or **Spring Data MongoDB** (spring-boot-starter-data-mongodb) for Spring Boot.
- Define a Student schema/document with the same fields used in Lab 3: name, email, course, semester.
- Store the Atlas connection string in an environment variable (.env for Express, application.properties for Spring Boot) - never hard-code or commit it.
- Rewrite all five CRUD operations to read/write MongoDB instead of the in-memory structure: GET /students, GET /students/{id}, POST /students, PUT/PATCH /students/{id}, DELETE /students/{id}.
- Keep the validation and HTTP status codes from Lab 3 working (400 for bad input, 404 for missing id).
- **Data integrity:** decide on at least one database constraint for the Student model - for example, whether **email** should be unique - and apply it as a MongoDB index/constraint. Briefly note your reasoning in the README.

Proof of persistence: add a student via Postman, stop and restart your server, then GET /students again. The data must still be there - this is the key evidence that the database is actually connected, not just the driver installed.

Scope note: Do not switch frameworks or redesign the API. Use the same Express.js or Spring Boot project from Lab 3 and the same resource design - only the storage layer changes, from in-memory to MongoDB Atlas.

4. Handling Cross-Origin Requests (CORS)

React runs on one origin (e.g. localhost:5173) and your API runs on another (e.g. localhost:3000). Browsers block cross-origin requests by default, so the API must explicitly allow the React app's origin - this is one of the most common integration problems you will hit in this lab.

- **Express.js:** install and enable the cors package - `npm install cors` then `app.use(cors());` before your routes.
- **Spring Boot:** allow the React origin with `@CrossOrigin(origins = "http://localhost:5173")` on your controller, or a global `CorsConfigurationSource` bean.
- Android is not affected by CORS (it is not a browser), but the API must still be reachable on the correct host/port.

5. API Base URL & Environment Configuration

Do not hard-code full URLs (e.g. `http://localhost:3000/students`) throughout your app. Define a single **API_BASE_URL** constant/config value and build every request path from it - this makes the client portable across environments.

Client	Typical Base URL Consideration
React (browser)	localhost refers to your own development machine
Android Emulator	10.0.2.2 refers to the host machine running the API
Physical Android Device	May need your machine's network IP instead of localhost
Cloud Deployment (later)	Public/cloud API URL, once the API is hosted online

Keep `API_BASE_URL` in one place (e.g. a `.env` value or a config file) in both the React and Android projects, so switching environments later means changing one line, not every API call.

6. Part B - React Web Client (Full CRUD) [Required in full]

Build a React application that fully consumes the Student REST API - this is your complete, primary client implementation for this lab.

- Scaffold a React app (Vite or Create React App) named **student-client**.
- Build a **StudentList** component that fetches and displays all students (GET /students) in a table or card layout.
- Build a **StudentForm** component used for both adding a new student (POST /students) and editing an existing one (PUT/PATCH /students/{id}).
- Add a Delete action on each student row (DELETE /students/{id}).
- Use `fetch()` or `axios`; manage state with `useState/useEffect`.
- After every add/edit/delete, refresh the list from the server so the UI always reflects real database state.
- Show loading and error states in the UI (e.g. "Loading students...", "Unable to load data. Please try again.").
- Add basic client-side validation before submit (empty name, invalid email format) mirroring the backend's validation rules.
- Translate specific HTTP responses into specific UI messages - e.g. a 400 from POST /students shows the validation error returned by the API; a 404 from GET /students/{id} shows "Student not found".
- Build all requests from **API_BASE_URL** - never hard-code the full URL inline.

7. Part C - Android Client (Equivalent Screens) [Partial, like Lab 3's second stack]

Build a small Android app that talks to the **same** Student REST API. You are not rebuilding the whole CRUD here - just enough to prove the API can serve a mobile client too.

- Create an Android Studio project (Kotlin or Java).
- Implement at least **two** equivalent operations:
 - **List Students** screen - calls GET /students and shows results in a RecyclerView/ListView.
 - **Add Student** screen - a simple form that calls POST /students.
- Use **Retrofit** (preferred) or HttpURLConnection/Volley for the network calls.
- If the API runs on your own machine and you test on the Android Emulator, use **10.0.2.2** in place of localhost/127.0.0.1 - keep this as your API_BASE_URL for the emulator.
- Show a Toast or on-screen message for success and failure responses, including a specific message for a 400 (validation error) vs a 404 (not found) response.

8. HTTP Error Handling - The Service Contract

HTTP status codes are part of the contract between your API and every client that consumes it. Both React and Android should react to the same codes in a client-appropriate way.

Request	Response	React Behaviour	Android Behaviour
POST /students (invalid body)	400 Bad Request	Show inline validation error near the form	Show Toast with the validation error
GET /students/999 (missing id)	404 Not Found	Show "Student not found" message	Show Toast: "Student not found"
Any request, server/network issue	5xx / network error	Show "Unable to load data. Please try again."	Show Toast: "Something went wrong"

9. Endpoints (Same Design as Lab 3, Now Backed by MongoDB)

Operation	Method	Endpoint	Used By	Expected Result
List students	GET	/students	React, Android	200 OK
Get one	GET	/students/{id}	React	200 OK / 404
Create	POST	/students	React, Android	201 Created
Update	PUT / PATCH	/students/{id}	React	200 OK / 404
Delete	DELETE	/students/{id}	React	204 / 200 OK

10. Step-by-Step Procedure

- **Step 1** — Confirm your Lab 3 Student API (Express.js or Spring Boot) still runs correctly with in-memory CRUD.
- **Step 2** — Create a MongoDB Atlas account, a free M0 cluster, a database user, and allow network access.
- **Step 3** — Copy the Atlas connection string (SRV URI) for your cluster.

- **Step 4** — Install the database driver: `npm install mongoose` (Express.js) or add the Spring Data MongoDB starter dependency (Spring Boot).
- **Step 5** — Define the Student model/document with the Lab 3 fields, decide on a constraint (e.g. unique email), and connect to Atlas at server startup using the connection string from your `.env` / `application.properties`.
- **Step 6** — Replace the in-memory logic in each of the five route handlers/controller methods with the equivalent MongoDB operation (`find`, `findById`, `save/create`, `findByIdAndUpdate`, `findByIdAndDelete`).
- **Step 7** — Enable CORS on the API (`app.use(cors())` for Express, `@CrossOrigin` / `CorsConfigurationSource` for Spring Boot) so the React app's origin is allowed.
- **Step 8** — Test in Postman: create a few students, restart the server, and confirm the data persisted.
- **Step 9** — Scaffold the React app; define `API_BASE_URL`; build `StudentList` and `StudentForm`; wire up all five endpoints with `fetch()`/`axios`.
- **Step 10** — Add loading/error states to the React app, including specific handling for 400 and 404 responses, and basic client-side validation.
- **Step 11** — Scaffold the Android app; define `API_BASE_URL` (10.0.2.2 for the emulator); implement the `List Students` and `Add Student` screens with Retrofit, including Toast messages for 400/404 responses.
- **Step 12** — Capture Postman, React and Android screenshots as evidence, then update `README.md` with the architecture diagram, the "why REST not direct DB access" answer, and setup/run instructions for all three parts.

11. Submission Requirements (What to Submit)

- Updated backend project from Lab 3, now connected to MongoDB Atlas (Mongoose or Spring Data MongoDB), with CORS enabled and `.env` / `application.properties` excluded from version control.
- Postman evidence: CRUD requests against the database, plus a screenshot/proof showing data persisted after a server restart.
- Complete React client project (student-client) source code implementing all five endpoints, using a single `API_BASE_URL` and handling 400/404 responses explicitly.
- Screenshots of the React app: student list, add form, edit flow, after a delete, and at least one error state (e.g. validation error or not-found message).
- Android project source code implementing `List Students` and `Add Student` screens, using `API_BASE_URL` and Toast messages for 400/404 responses.
- Screenshots (or short screen recording) of the Android app running on an emulator or device, showing the list loading and a new student being added.
- Updated `README.md` containing: a full-stack architecture diagram (React + Android + REST API + MongoDB), your answer to "why REST and not direct MongoDB access", the database constraint you chose and why, and setup/run instructions for the backend, React client and Android app.
- A short written reflection (5-8 lines) comparing consuming the same REST API from a web client versus a mobile client - what stayed the same, and what had to be handled differently.

12. Learning Outcomes

By the end of this laboratory, students will be able to: migrate an in-memory REST service to persistent MongoDB Atlas storage without changing its API contract; develop a React client implementing CRUD operations; develop an Android client consuming the same REST API; configure client-server communication across different execution environments; handle loading and API errors; and explain how multiple independent clients can consume a common REST service.

13. Final Checklist

Database	React Client	Android Client	Concepts & Evidence
MongoDB Atlas cluster created	StudentList (GET all)	List Students screen (GET)	Postman screenshots
Student schema/model + constraint (e.g. unique email)	StudentForm (add/edit)	Add Student screen (POST)	Persistence-after-restart proof
Connection string in env file	Delete action wired up	Retrofit / networking configured	React + Android app screenshots
All 5 CRUD routes use MongoDB	API_BASE_URL used, no hard-coded URLs	API_BASE_URL used (10.0.2.2 for emulator)	CORS enabled and working
Status codes 200/201/400/404 kept	400/404 handled with specific UI messages	Toast on 400/404/success	"Why REST not direct DB" answer in README

Important: Keep the resource design (fields, endpoints, status codes) identical to Lab 3. This lab adds a persistence layer and two client applications - it does not redesign the API.